HRMS

Human Resource Management System

Project Report

Name : Adhiraj Deshmukh

Date : 18-03-2026

Batch Name : WIPRO NGA - .Net full Stack React – FY26

Project Name	HR Management System (HRMS)
Frontend	React.js with Vite
Backend	ASP.NET Core Web API (.NET 8) — C#
Database	Microsoft SQL Server (MSSQL LocalDB)
API Documentation	Swagger UI (OpenAPI)
Frontend Port	localhost:5173
Backend Port	localhost:5230

1. Introduction

HRMS (Human Resource Management System) is a full-stack web application designed to streamline and automate HR operations within an organization. It provides a centralized platform for managing employees, processing leave requests, and gaining analytical insights into the workforce.

The system supports two types of users: HR Administrators who manage employee data and leave approvals, and Employees who can apply for leave and track their leave history. The application is built using modern web technologies to ensure a responsive, role-based user experience backed by a robust RESTful API.

Without a structured system, HR processes become inefficient and error-prone. HRMS solves this by offering a digital, structured solution that brings all HR functions under one roof — from onboarding employees to approving leave requests.

2. Objectives

The main objectives of this project are to:

- Provide a secure role-based login system for HR and Employee users
- Allow HR administrators to add, edit, and delete employee records
- Enable employees to apply for leave and track their leave application status
- Allow HR to view, approve, or reject employee leave requests
- Display a dashboard with analytics showing total employees and total leaves
- Expose a documented REST API through Swagger UI for all operations
- Persist all data in a relational SQL Server database
- Support analytics endpoints for department headcount and workforce overview

3. System Architecture

HRMS follows a classic three-tier architecture. Each layer is independently responsible for a specific concern, ensuring clean separation of responsibilities and making the system maintainable and scalable.

3.1 Frontend Layer

Built using React.js with Vite as the build tool, the frontend is a single-page application (SPA). Axios is used as the HTTP client to communicate with the backend at localhost:5230/api. React Router DOM manages client-side routing with role-based navigation.

3.2 Backend Layer

Developed in C# using ASP.NET Core Web API (.NET 8), the backend processes all business logic and serves RESTful endpoints. It is structured into Controllers, Services, and Data layers. Swagger is integrated for API documentation. CORS is configured to allow the React frontend to communicate cross-origin.

3.3 Database Layer

Microsoft SQL Server (MSSQL LocalDB) stores all application data in the HRManagementDB database. Entity Framework Core is used via ApplicationDbContext for ORM-based access to two primary tables: Employees and LeaveRequests.

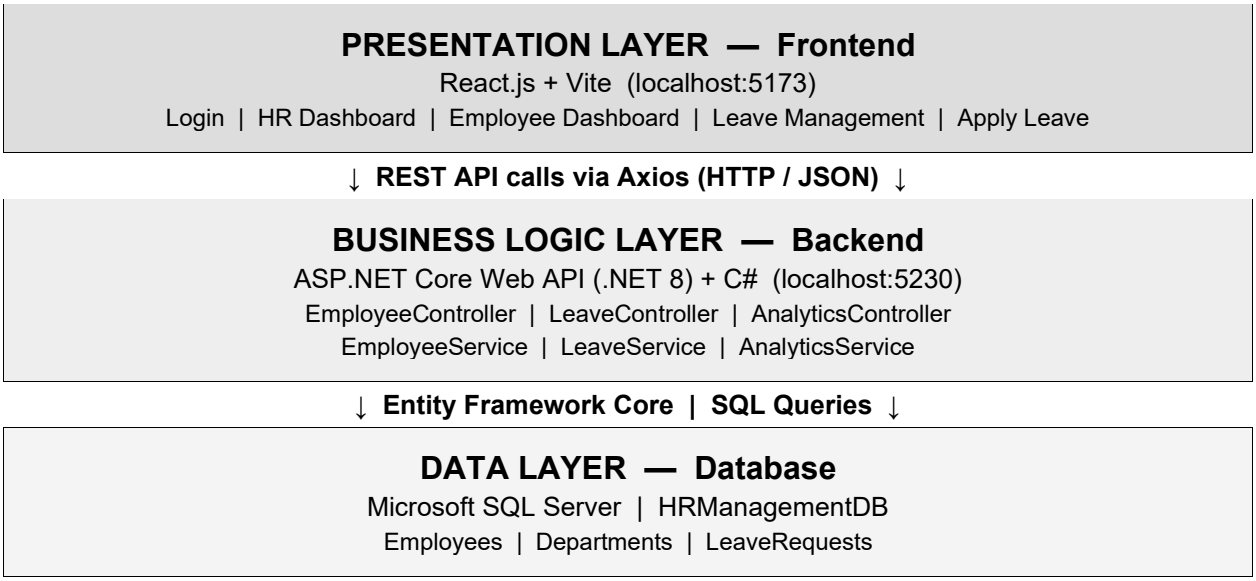

Figure 1: Three-Tier System Architecture Diagram

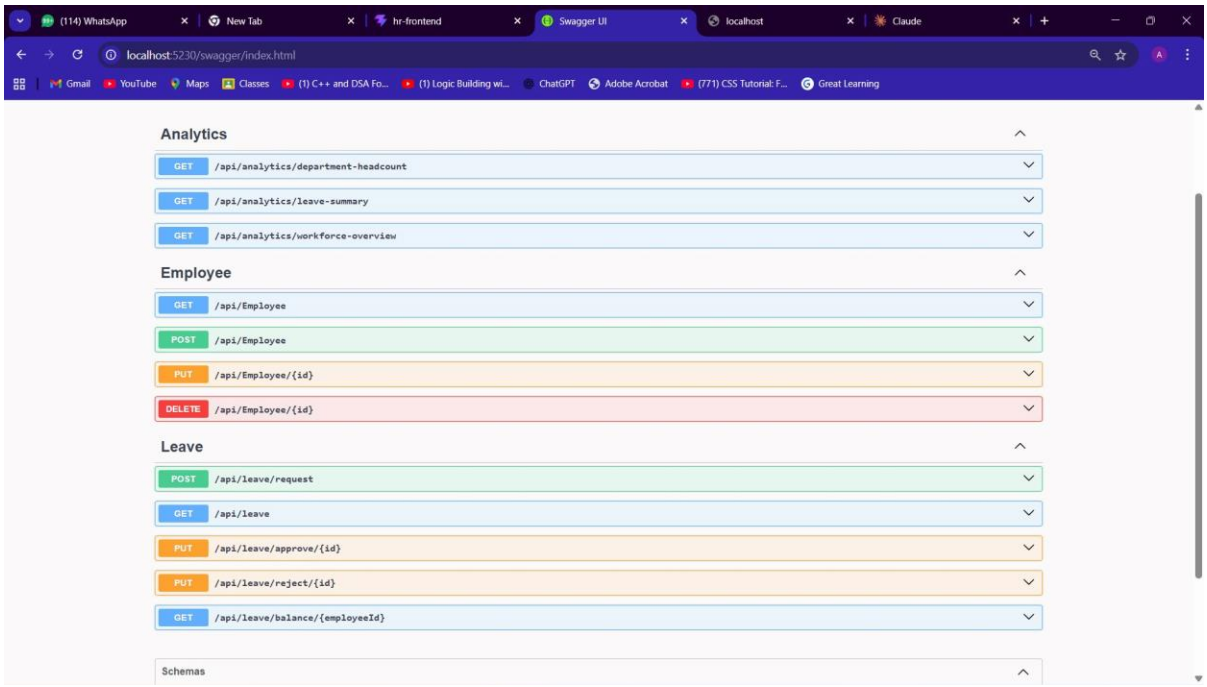

Figure 2: Swagger UI showing all available REST API endpoints

4. Data Flow & Application Flow

4.1 Application User Flow

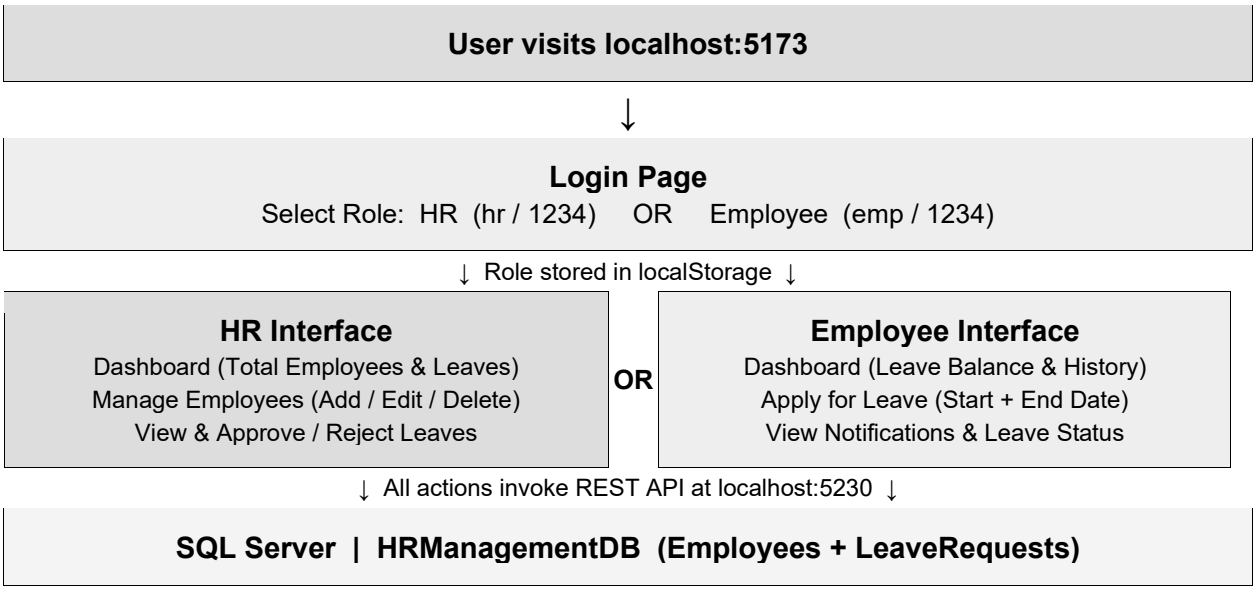

Figure 3: Application User Flow Diagram

4.2 Leave Request Data Flow

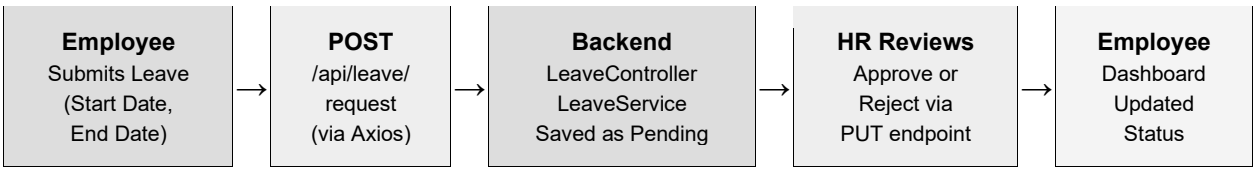

Figure 4: Leave Request Data Flow

5. Database Design

The HRManagementDB database follows normalization principles up to the Third Normal Form (3NF). Primary keys ensure unique identification and foreign keys maintain referential integrity between related tables.

5.1 Tables

Table	Primary Key	Columns	Description
Employees	EmployeeId (INT PK)	FirstName, LastName, Email, DeptId (FK), LeaveBalance	Stores all employee records
Departments	DepartmentId (INT PK)	DepartmentName (VARCHAR)	HR, IT, Finance
LeaveRequests	LeaveId (INT PK)	EmployeeId (FK), StartDate, EndDate, Status (VARCHAR 20)	Tracks leave lifecycle

5.2 Entity Relationship (ER) Diagram

Figure 5: Entity Relationship (ER) Diagram

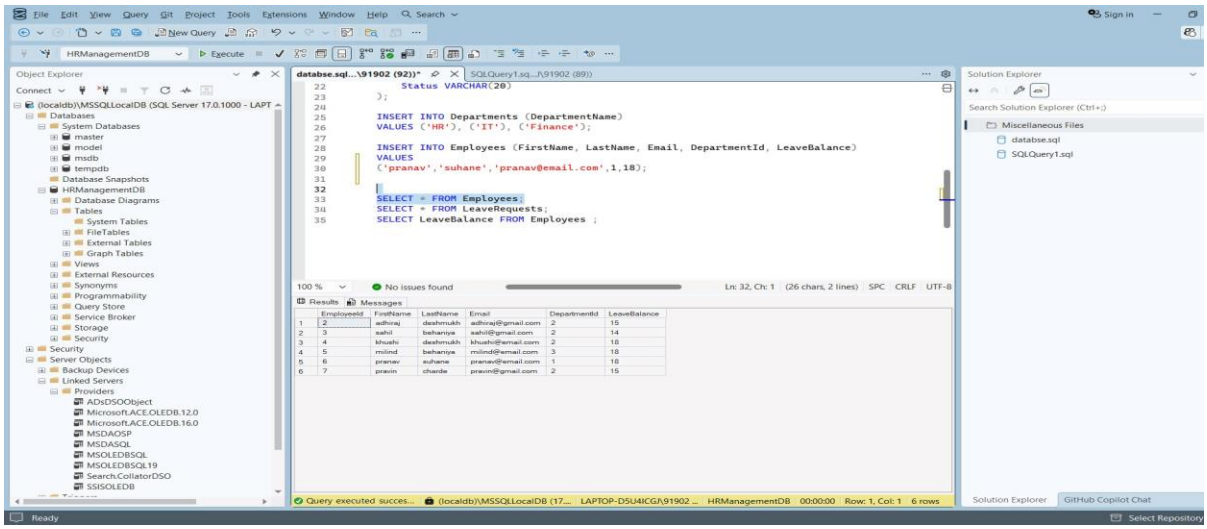

Figure 6: SQL Server Management Studio — HRManagementDB with sample employee records

6. Backend API Documentation

The backend exposes REST API endpoints organized into three groups: Employee, Leave, and Analytics. All endpoints are accessible under localhost:5230/api and fully documented through Swagger UI at localhost:5230/swagger/index.html.

6.1 Employee Endpoints

Method	Endpoint	Description
GET	`/api/Employee`	Retrieve all employee records
POST	`/api/Employee`	Add a new employee
PUT	`/api/Employee/{id}`	Update employee details by ID
DELETE	`/api/Employee/{id}`	Remove employee by ID

6.2 Leave Endpoints

Method	Endpoint	Description
POST	`/api/leave/request`	Submit a new leave request
GET	`/api/leave`	Retrieve all leave requests
PUT	`/api/leave/approve/{id}`	Approve a leave request by ID
PUT	`/api/leave/reject/{id}`	Reject a leave request by ID
GET	`/api/leave/balance/{employeeId}`	Get leave balance for an employee

6.3 Analytics Endpoints

Method	Endpoint	Description
GET	`/api/analytics/department-headcount`	Employee count per department
GET	`/api/analytics/leave-summary`	Summary of approved/rejected leaves
GET	`/api/analytics/workforce-overview`	Overall workforce statistics

7. Frontend Implementation

The React frontend is a single-page application (SPA) with role-based routing. The App.jsx component reads the user role from localStorage and conditionally renders the HR or Employee interface. All API calls are centralized in src/api/api.js using an Axios instance configured with the backend base URL.

7.1 Login Page

The login page features a split-panel layout. The left panel contains the HRMS branding, a role toggle (HR / Employee), username and password inputs, and a Sign In button. The right panel displays a contextual illustration with a caption that changes based on the selected role. Credentials are validated client-side and the role is stored in localStorage.

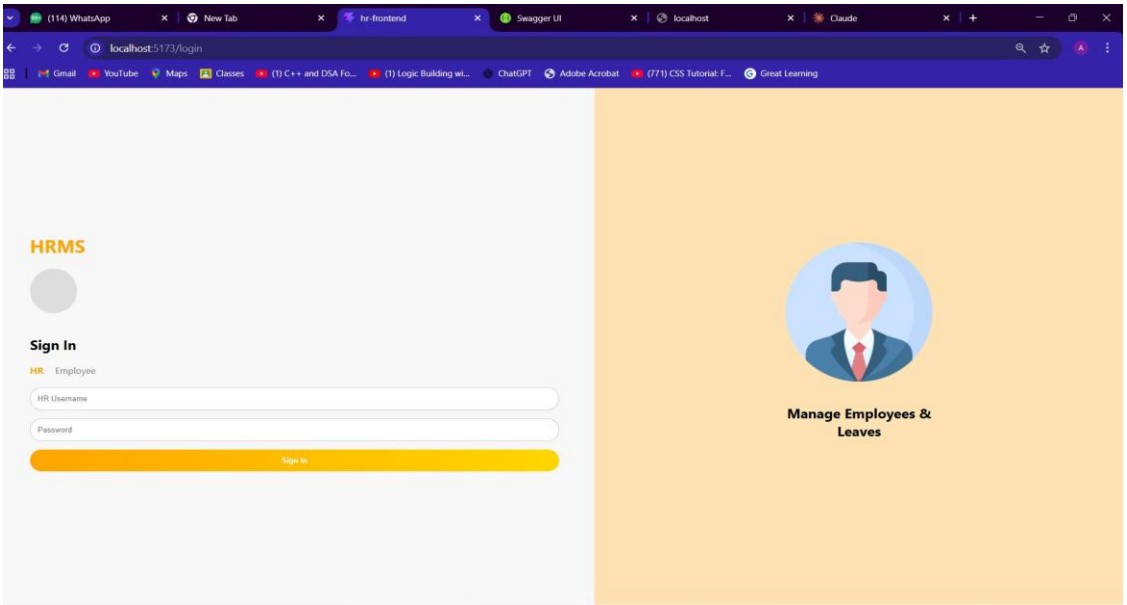

Figure 7: Login Page — HR role selected, showing 'Manage Employees & Leaves'

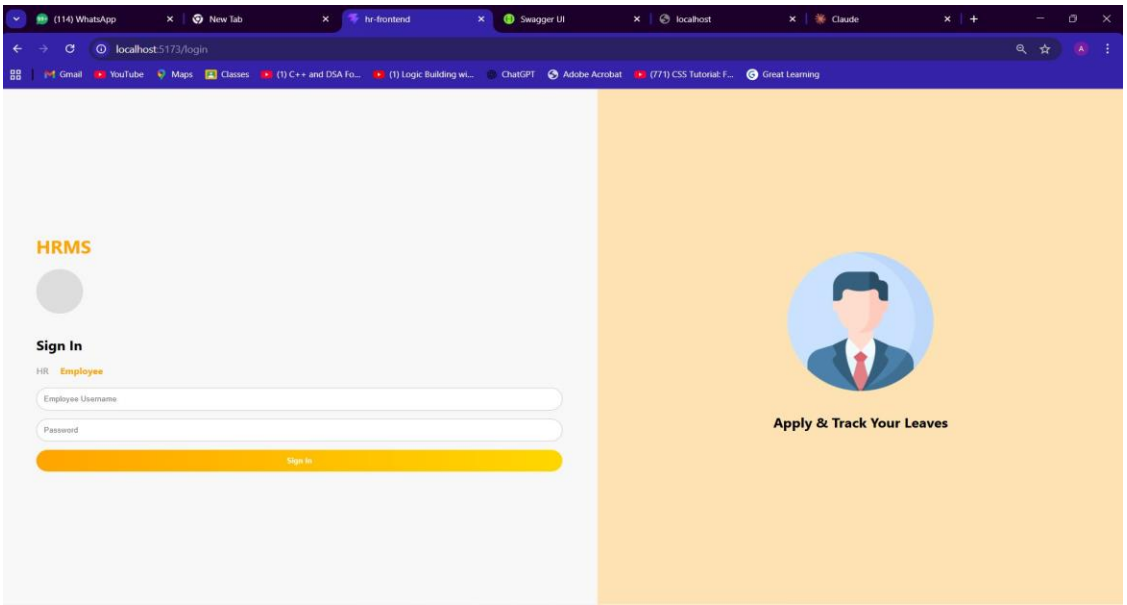

Figure 8: Login Page — Employee role selected, showing 'Apply & Track Your Leaves'

7.2 HR Dashboard

After HR login, the dashboard displays two summary stat cards: Total Employees and Total Leaves. Data is fetched from the Analytics and Leave API endpoints on component mount. The sidebar provides navigation links to Dashboard, Employees, and Leaves sections.

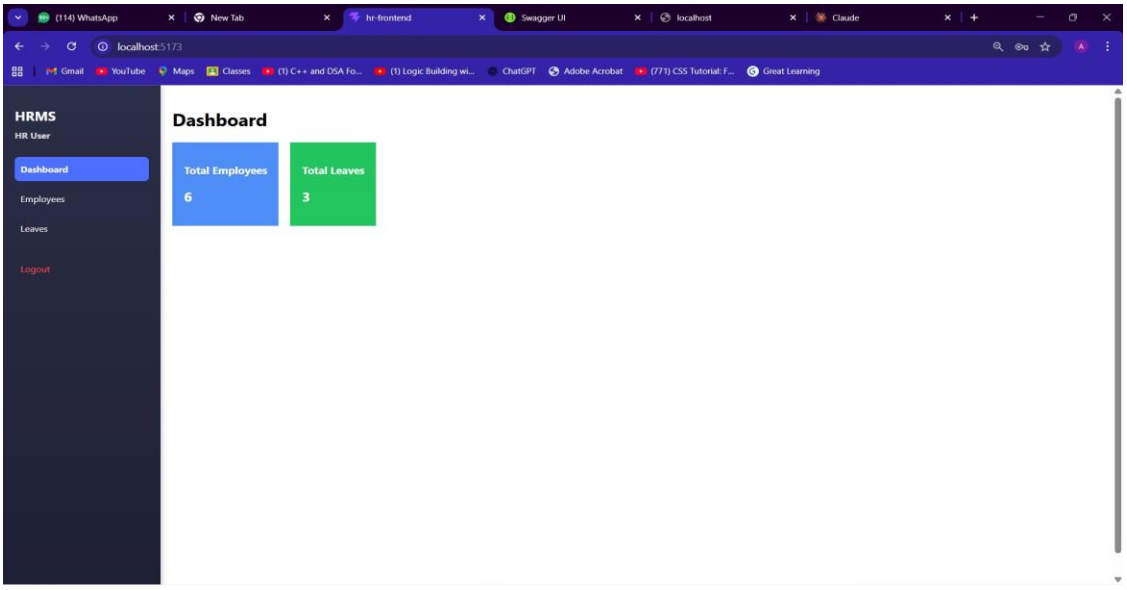

Figure 9: HR Dashboard — Total Employees: 6, Total Leaves: 3

7.3 Employee Management Page

The Employees page lists all employees in a structured table showing ID, Full Name, Email, Department ID, Leave Balance, and Action buttons. A form at the top allows HR to add new employees. Each row has Edit and Delete action buttons for in-place record management.

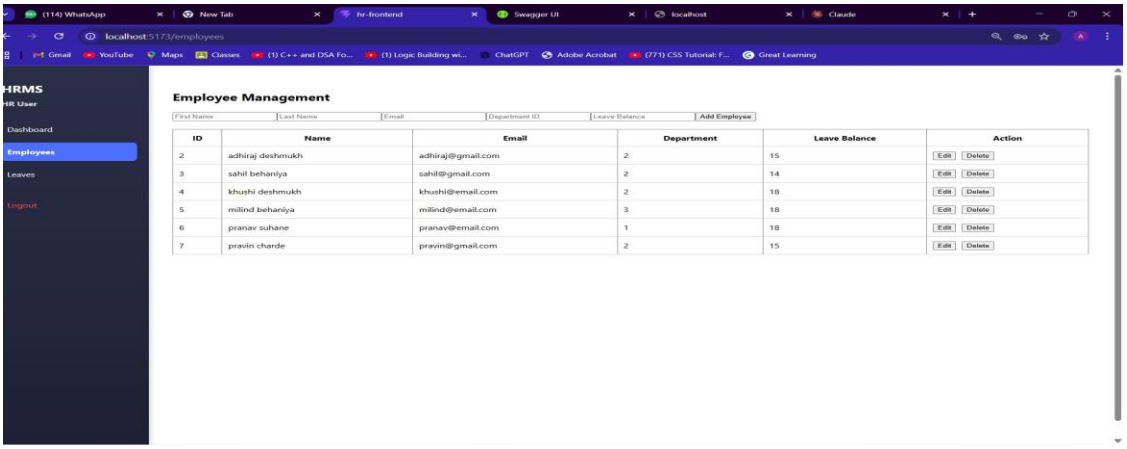

Figure 10: Employee Management — CRUD table with Add Employee form

7.4 Leave Requests Page (HR View)

The Leaves page shows all leave requests with columns for ID, Employee ID, Start Date, End Date, Status, and Actions. Pending requests display Approve and Reject buttons. Already-processed requests show only their final status label.

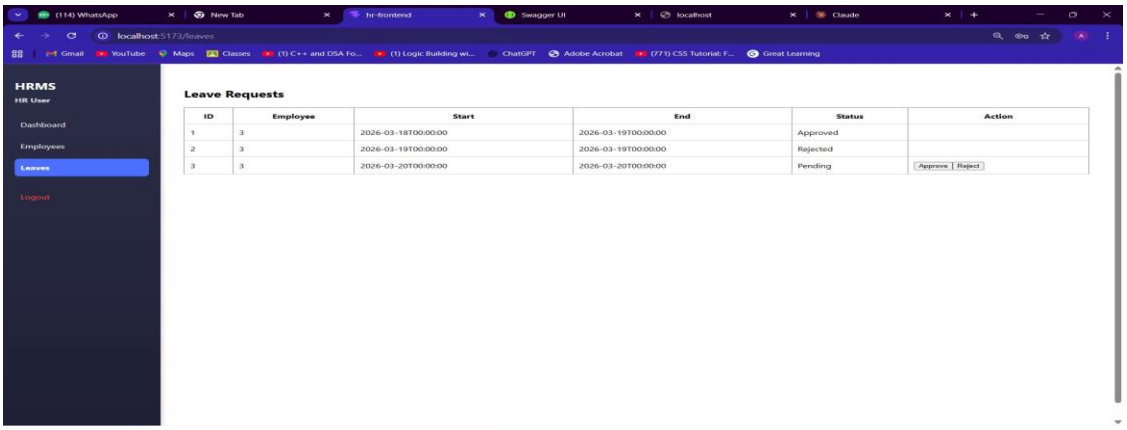

Figure 11: HR Leave Requests — Approve/Reject actions for pending requests

7.5 Employee Dashboard

The Employee dashboard greets the user by name and shows Leave Balance and Total Leaves stat cards. A Leave History table lists all requests with status. A Notifications section mirrors leave statuses as readable messages for the employee.

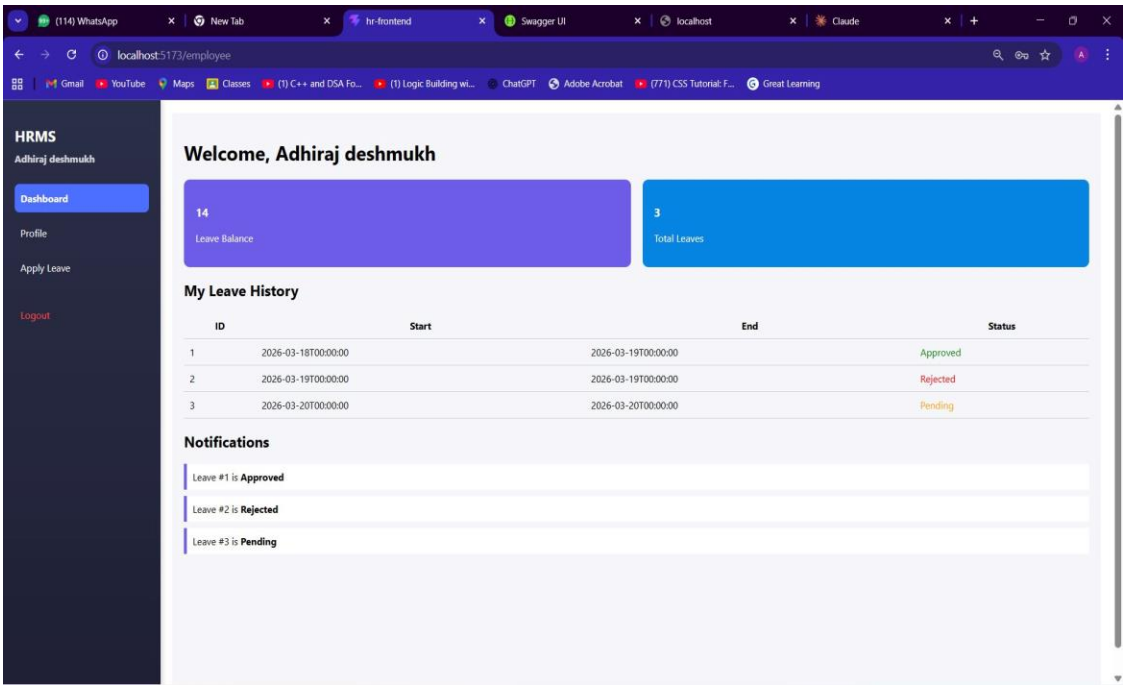

Figure 12: Employee Dashboard — Leave history with status indicators and notifications

7.6 Apply Leave Page

Employees can submit a new leave application by selecting a Start Date and End Date, then clicking Apply Leave. The form sends a POST request to /api/leave/request with the employee ID from localStorage. The new request appears on the dashboard with a Pending status.

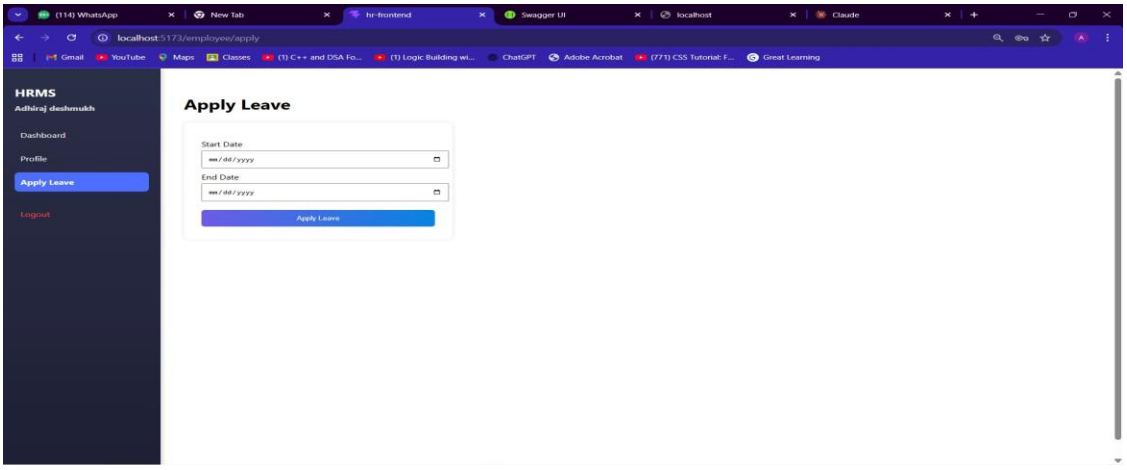

Figure 13: Apply Leave — Date picker form for submitting leave requests

8. Technology Stack

Category	Technology	Purpose
Frontend Framework	React.js 18 + Vite	SPA development and fast build tooling
Routing	React Router DOM	Client-side navigation and role-based routes
HTTP Client	Axios	REST API communication with backend
Backend Framework	ASP.NET Core Web API	RESTful API development and business logic
Backend Language	C# (.NET 8)	Controller, Service, and Model implementation
ORM	Entity Framework Core	Database access via ApplicationDbContext
Database	MS SQL Server (LocalDB)	Relational data persistence
API Documentation	Swagger (OpenAPI)	Interactive endpoint testing and documentation
Styling	CSS Modules / Plain CSS	Component-level scoped styling
BI / Analytics	Microsoft Power BI	Visual reporting and data insights

9. Module Breakdown

9.1 Authentication Module

Login is handled on the client side. The Login.jsx component validates credentials against hardcoded values (HR: hr/1234, Employee: emp/1234). On success, the user role is saved to localStorage. App.jsx reads this on every render to determine which interface to show, redirecting unauthenticated users to the login page.

9.2 Employee Management Module

Implemented through EmployeeController and EmployeeService on the backend, this module supports the full CRUD lifecycle. The Employees.jsx page re-fetches data after every add, edit, or delete operation to keep the UI in sync with the database.

9.3 Leave Management Module

Managed by LeaveController and LeaveService, this module handles the complete leave lifecycle: request submission (POST), listing all requests (GET), approval (PUT), and rejection (PUT). Leave balance is tracked per employee in the Employees table. EmployeeDashboard.jsx displays leave history with status indicators and a notifications list.

9.4 Analytics Module

AnalyticsController exposes three GET endpoints providing department headcount, a leave approval/rejection summary, and a workforce overview. These are designed for the HR dashboard and can be extended to feed charting libraries for richer data visualization.

10. Power BI Reports

Power BI is used to visualize the HRMS data and generate insights into employee distribution, leave trends, and overall workforce health. Data is obtained directly from the SQL Server HRManagementDB database and presented through KPI cards, pie charts, and summary tables.

10.1 Dashboard Overview

The Power BI report contains the following visuals on a single dashboard page:

- Employees Count KPI Card — shows the total number of active employees (6)
- Apply Leave KPI Card — shows the total number of leave requests submitted (3)
- Count of LeaveId by Status — a pie chart showing Approved, Pending, and Rejected leaves, each at 33.33% (1 each)
- Employee Summary Table — lists all employees with EmployeeId, FirstName, LastName, Email, Count of DepartmentId, and Sum of LeaveBalance (Total: 98)

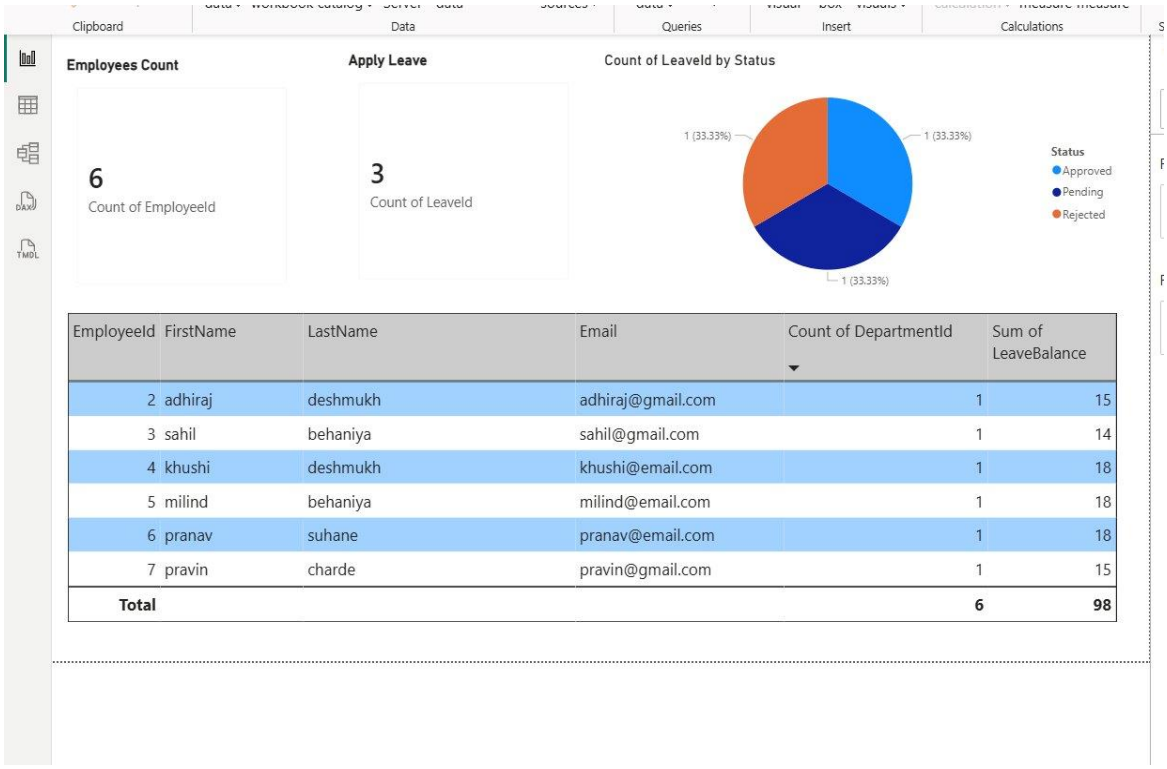

Figure 14: Power BI Dashboard — Employee count, leave status distribution, and employee summary table

10.2 Key Insights

Metric	Value / Insight
Total Employees	6 active employees across departments
Total Leave Requests	3 requests submitted
Leave Status Distribution	Approved: 1 (33.33%) | Pending: 1 (33.33%) | Rejected: 1 (33.33%)
Total Leave Balance (All Employees)	98 combined leave days across all 6 employees
Highest Leave Balance	Khushi Deshmukh, Milind Behaniya, Pranav Suhane — 18 days each
Lowest Leave Balance	Sahil Behaniya — 14 days

11. Conclusion

The HRMS project successfully demonstrates a complete full-stack application for automating core human resource operations. By integrating a React.js frontend, an ASP.NET Core Web API backend, and a Microsoft SQL Server database, the system delivers a reliable, role-based solution for employee and leave management.

The application cleanly separates concerns across its three tiers, making it maintainable and extensible. The RESTful API design, documented via Swagger, ensures the backend can serve any future client including mobile applications or third-party integrations. Power BI integration further enriches the system with data-driven insights for HR decision-making.

Potential future enhancements include JWT-based authentication for production-grade security, an expanded analytics dashboard with interactive chart visualizations, email notifications for leave status updates, pagination for large employee lists, and multi-department performance reporting.

— End of Report —